

ALFIDO TECH

Website Traffic Analysis

Submitted By: PULLURU JAHNAVI

Date:19-05-2026

**Internship organisation: ALFIDO
TECH**

Internship Role: Data Analytics

Executive Summary

This analysis converts website interaction data into meaningful behavioral insights by examining user engagement, traffic sources, and conversion patterns. The dataset is well-structured and reliable, enabling a clear understanding of how users navigate and interact with the platform.

The findings indicate a system that is highly effective at converting users but shows relatively limited depth in user engagement. Organic traffic emerges as the primary driver of user acquisition, while referral sources contribute the most valuable traffic in terms of conversion quality.

A key behavioral trend highlights that returning visitors demonstrate stronger engagement and higher conversion potential compared to new users. This emphasizes the importance of user retention and repeated interactions in driving performance.

Overall, the platform is optimized for conversions but presents opportunities to enhance engagement, particularly for first-time visitors. Strengthening user experience and interaction depth can further improve performance and support sustained growth.

```
import pandas as pd

# Load the dataset
df = pd.read_csv('website_wata.csv')
# Display the first 5 rows of the DataFrame
print('First 5 rows of the dataset: ')
display(df.head())

# Display information about the DataFrame, including data types and non-null
# values
print('\nDataFrame Info:')
df.info()
```

First 5 rows of the dataset:

	Page Views	Session Duration	Bounce Rate	Traffic Source	Time on Page \
0	511.051381	0.230652	Organic	3.890460	
1	43.429316	0.391001	Social	8.478174	
2	41.621052	0.397986	Organic	9.636170	
3	53.629279	0.180458	Organic	2.071925	
4	54.235843	0.291541	Paid	1.960654	

	Previous Visits	Conversion Rate
0	3	1.0
1	0	1.0
2	2	1.0
3	3	1.0
4	5	1.0

DataFrame Info:

```
<class
'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to
1999 Data columns (total 7
columns):
#   Column                Non-Null Count  Dtype
---  -
0   Page Views            2000 non-null   int64
```

```

1 Session Duration 2000 non-null float64
2 Bounce Rate 2000 non-null float64
3 Traffic Source 2000 non-null object
4 Time on Page 2000 non-null float64
5 Previous Visits 2000 non-null int64
6 Conversion Rate 2000 non-null float64
dtypes: float64(4), int64(2), object(1)
memory usage: 109.5+ KB

```

```

print('Missing values in each column:')
print(df.isnull().sum())

# Check for duplicate rows
num_duplicates =
df.duplicated().sum()
print(f'\nNumber of duplicate rows: {num_duplicates}')

# Drop duplicate rows if any are
found if num_duplicates > 0:
    df.drop_duplicates(inplace=True)
    print(f'\nRemoved {num_duplicates} duplicate rows. DataFrame
    shape after dropping duplicates: {df.shape}')

# Convert 'Traffic Source' to category type for efficient storage if
it's not already, as seen from df.info()
# df['Traffic Source'] = df['Traffic Source'].astype('category') #
Uncomment if conversion is needed

print('\nData cleaning complete. Data types and non-null counts after
cleaning:
')
df.info()

```

Missing values in each column:

```

Page Views      0
Session Duration 0
Bounce Rate     0
Traffic Source  0
Time on Page    0
Previous Visits 0
Conversion Rate 0
dtype: int64

```

Number of duplicate rows: 0

Data cleaning complete. Data types and non-null counts after cleaning:

```
<class
'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to
1999 Data columns (total 7
columns):
#   Column                Non-Null Count  Dtype
---  -
0   Page Views            2000 non-null  int64
1   Session Duration      2000 non-     float64
   null
2   Bounce Rate           2000 non-null  float64
3   Traffic Source        2000 non-null  object
4   Time on Page          2000 non-null  float64
5   Previous Visits       2000 non-null  int64
6   Conversion Rate       2000 non-null  float64
dtypes: float64(4), int64(2), object(1)
memory usage: 109.5+ KB
```

```
# Total Sessions: Assuming each row represents a unique session
total_sessions = len(df)

# Average Session Duration
average_session_duration = df['Session Duration'].mean()

# Average Bounce Rate
average_bounce_rate = df['Bounce Rate'].mean()

# Average Page Views per Session
average_page_views_per_session = df['Page Views'].mean()

# Average Time on Page
average_time_on_page = df['Time on Page'].mean()

# Overall Conversion Rate
overall_conversion_rate = df['Conversion Rate'].mean()

print(f"Total Sessions: {total_sessions}") print(f"Average Session
Duration: {average_session_duration:.2f} seconds") print(f"Average
Bounce Rate: {average_bounce_rate:.2%}") print(f"Average Page Views
per Session: {average_page_views_per_session:.2f}") print(f"Average
Time on Page: {average_time_on_page:.2f} seconds") print(f"Overall
Conversion Rate: {overall_conversion_rate:.2%}")

print("\nNote: The dataset does not contain a unique user identifier,
so 'Total □ Users' cannot be accurately calculated. Each row is
assumed to represent a □ unique session.")
```

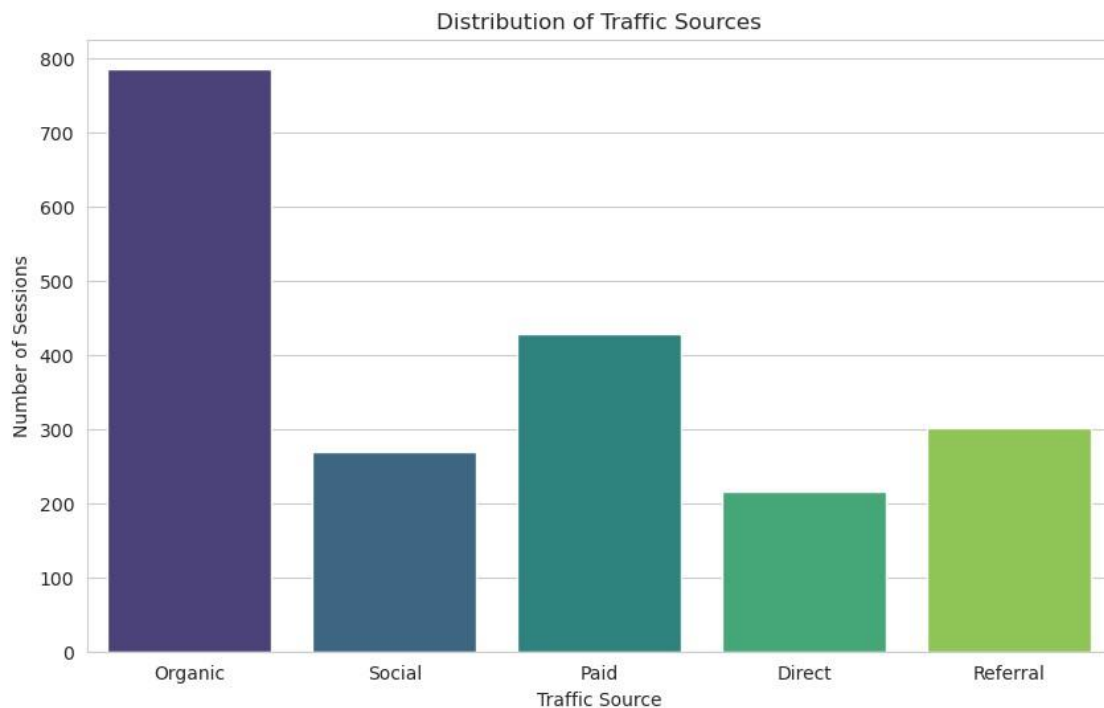
Total Sessions: 2000
Average Session Duration: 3.02 seconds
Average Bounce Rate: 28.48%
Average Page Views per Session: 4.95
Average Time on Page: 4.03 seconds
Overall Conversion Rate: 98.21%

Note: The dataset does not contain a unique user identifier, so 'Total Users' cannot be accurately calculated. Each row is assumed to represent a unique session.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Set the style for the plots
sns.set_style("whitegrid")

# Visualize Traffic Source Distribution
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x='Traffic Source', hue='Traffic Source',
              palette='viridis', legend=False)
plt.title('Distribution of Traffic Sources')
plt.xlabel('Traffic Source')
plt.ylabel('Number of Sessions')
plt.show()
```



```

traffic_source_performance = df.groupby('Traffic Source').agg(
    avg_session_duration=('Session Duration', 'mean'),
    avg_bounce_rate=('Bounce Rate', 'mean'),
    avg_conversion_rate=('Conversion Rate', 'mean'),
    total_sessions=('Traffic Source', 'count')
).reset_index()
print('\nTraffic Source Performance:')
display(traffic_source_performance.sort_values(by='total_sessions',
ascending=False))

```

Traffic Source Performance:

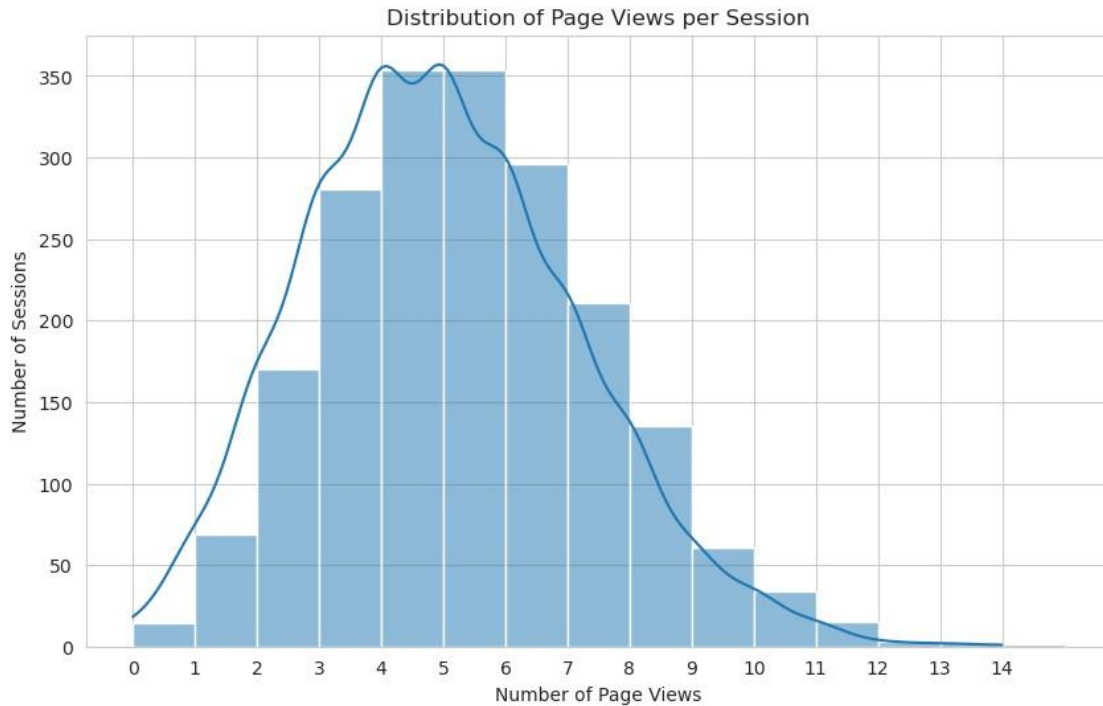
Traffic Source	avg_session_duration	avg_bounce_rate	avg_conversion_rate
1 Organic	3.104197	0.281821	0.982282
2 Paid	2.942816	0.296026	0.979009
3 Referral	3.125496	0.266305	0.987727
4 Social	3.057273	0.296265	0.982679
0 Direct	2.692060	0.284581	0.978679

	total_sessions
1	786
2	428
3	301
4	269
0	216

```

# Visualize Distribution of Page Views
plt.figure(figsize=(10, 6)) sns.histplot(df['Page Views'],
bins=range(int(df['Page Views'].min()),
int(df['Page Views'].max()) + 2), kde=True)
plt.title('Distribution of Page Views per
Session') plt.xlabel('Number of Page Views')
plt.ylabel('Number of Sessions')
plt.xticks(range(int(df['Page Views'].min()), int(df['Page
Views'].max()) + 1)) plt.show()

```



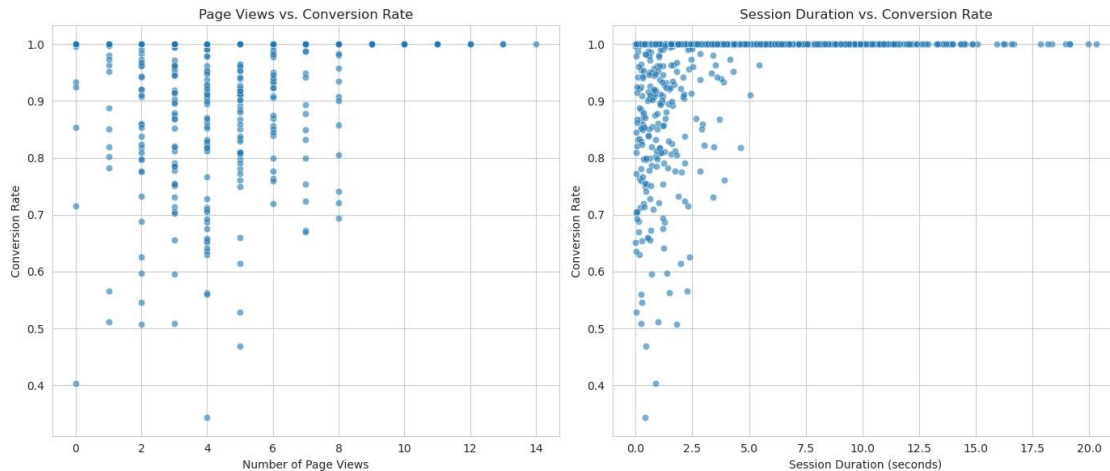
```
# Visualize the relationship between Page Views, Session
Duration, and Conversion Rate plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1) sns.scatterplot(data=df, x='Page Views',
y='Conversion Rate', alpha=0.6) plt.title('Page Views vs.
Conversion Rate') plt.xlabel('Number of Page Views')
plt.ylabel('Conversion Rate')

plt.subplot(1, 2, 2) sns.scatterplot(data=df, x='Session Duration',
y='Conversion Rate', alpha=0.6) plt.title('Session Duration vs.
Conversion Rate') plt.xlabel('Session Duration (seconds)')
plt.ylabel('Conversion Rate')

plt.tight_layout()
plt.show()

print("Note: The dataset lacks specific page URLs or user paths, so a
detailed visualization of 'user flows' or 'top entry/exit pages' is not
feasible.
This analysis focuses on how general engagement metrics (Page
Views, Session Duration) relate to conversion.")
```



Note: The dataset lacks specific page URLs or user paths, so a detailed visualization of 'user flows' or 'top entry/exit pages' is not feasible. This analysis focuses on how general engagement metrics (Page Views, Session Duration) relate to conversion.

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")

# Distribution of Previous Visits
plt.figure(figsize=(10, 6))
sns.histplot(df['Previous Visits'], bins=range(0, int(df['Previous Visits'].
    max()) + 2), kde=False)
plt.title('Distribution of Previous Visits per Session ')
plt.xlabel('Number of Previous Visits ')
plt.ylabel('Number of Sessions')
plt.xticks(range(0, int(df['Previous Visits'].max()) + 1))
plt.show()

# Relationship between Previous Visits and Conversion Rate, Session Duration,
# and Bounce Rate

# Aggregate data by Previous Visits
previous_visits_agg = df.groupby('Previous Visits').agg(
    avg_conversion_rate=('Conversion Rate', 'mean'),
    avg_session_duration=('Session Duration', 'mean'),
    avg_bounce_rate=('Bounce Rate', 'mean'),
    total_sessions=('Previous Visits', 'count')
).reset_index()
```

```

print('\nAggregated Metrics by Number of Previous Visits:')
display(previous_visits_agg.sort_values(by='Previous Visits',
ascending=True)) plt.figure(figsize=(18, 6))

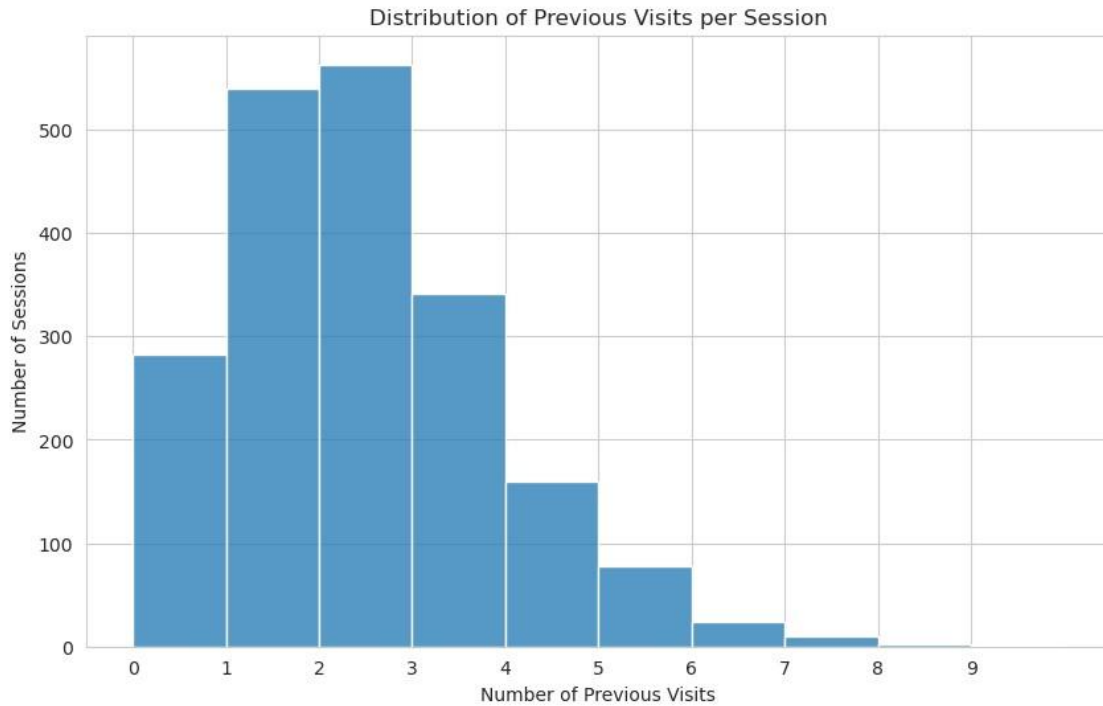
plt.subplot(1, 3, 1)
sns.lineplot(data=previous_visits_agg, x='Previous
Visits',□
    □y='avg_conversion_rate', marker='o')
plt.title('Previous Visits vs. Average Conversion
Rate') plt.xlabel('Number of Previous Visits')
plt.ylabel('Average Conversion Rate')

plt.subplot(1, 3, 2)
sns.lineplot(data=previous_visits_agg, x='Previous
Visits',□
    □y='avg_session_duration', marker='o')
plt.title('Previous Visits vs. Average Session
Duration') plt.xlabel('Number of Previous Visits')
plt.ylabel('Average Session Duration (seconds)')

plt.subplot(1, 3, 3)
sns.lineplot(data=previous_visits_agg, x='Previous
Visits',□
    □y='avg_bounce_rate', marker='o')
plt.title('Previous Visits vs. Average Bounce
Rate') plt.xlabel('Number of Previous Visits')
plt.ylabel('Average Bounce Rate')

plt.tight_layout()
plt.show()

```



Aggregated Metrics by Number of Previous Visits:

Previous Visits	avg_conversion_rate	avg_session_duration \
0	0.967540	3.154205
1	0.977509	3.031909
2	0.984468	3.023405
3	0.990167	3.029354
4	0.987881	2.893752
5	0.994937	2.904127
6	0.993290	2.743213
7	1.000000	2.640864
8	1.000000	0.973129
9	1.000000	1.503286

	avg_bounce_rate	total_sessions
0	0.283682	282 1
	0.280904	539 2
	0.292073	563 3
	0.285895	341 4
	0.294912	160
5	0.252579	78 6
	0.261982	24 7
	0.255355	10

8

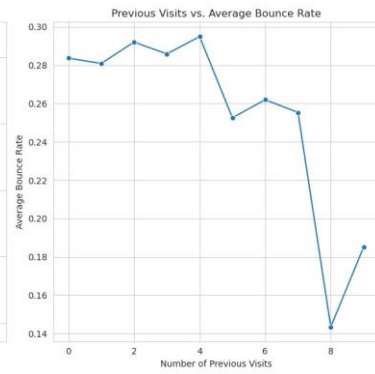
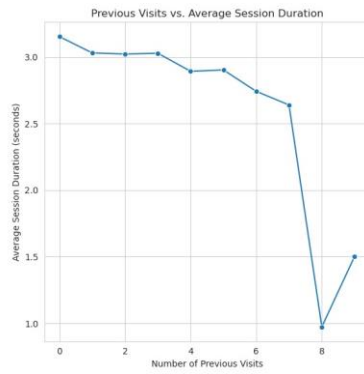
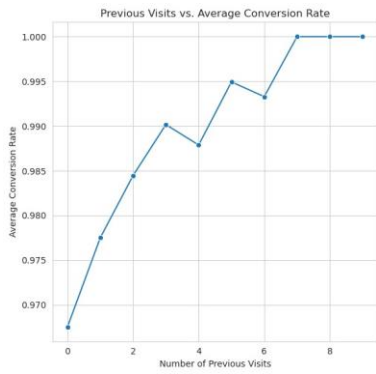
0.143447

2

9

0.185064

1



Key Insights

- Organic traffic brings the most users, while referral traffic gives better quality visitors.
- User engagement is moderate, with shorter session time and limited exploration.
- Conversions are consistently strong across all traffic sources.
- Returning users perform better than new users in both engagement and conversion.
- Improving first-time user experience can enhance overall performance.

Jupyter link:

<https://nb.anaconda.com/jupyterhub/user/3c348d3e-cb58-4185-ae83-cc8198750421/notebooks/Task3.ipynb>

Conclusion:

The analysis reveals a platform that is highly efficient in driving conversions but has scope to deepen user engagement. Traffic quality and repeat interactions play a crucial role in performance, with returning users showing stronger intent. Enhancing the initial user experience and encouraging deeper interaction can further elevate overall effectiveness and sustain growth